A practical lab-based assessment of web application vulnerabilities using DVWA, focusing on reconnaissance, analysis, and controlled exploitation.

DVWA Pentest Lab

Practical Hands-On Web Application Security Assessment

MANUEL MOLAPO

Part 1

Information Gathering and Surface Mapping of a Local Web Server (DVWA Lab Setup)

Introduction:

The reconnaissance phase is the first and one of the most critical stages of a penetration test. Its primary objective is to gather as much information as possible about the target system in a controlled and legal environment. In this project, reconnaissance is performed on a locally hosted vulnerable web application, DVWA (Damn Vulnerable Web Application), deployed using the XAMPP environment.

This phase focuses on identifying the structure, technologies, and potential entry points of the target system without directly exploiting any vulnerabilities. Both passive and active reconnaissance techniques are used to build a comprehensive understanding of the application.

A variety of tools and techniques are employed, including:

- Manual content discovery through files such as robots.txt, sitemap.xml, and HTTP headers.
- Open-Source Intelligence (OSINT) techniques to gather publicly available information.
- Network scanning using Nmap to identify open ports and services
- Web server analysis using Nikto.

In addition, subdomain enumeration is introduced as a key reconnaissance technique. Subdomain enumeration is the process of identifying valid subdomains associated with a target domain. This is important because it expands the attack surface, potentially revealing hidden applications, services, or entry points that may contain vulnerabilities.

Two primary subdomain enumeration methods are explored:

- Brute Force – systematically guessing subdomains using wordlists
- Open-Source Intelligence (OSINT) – collecting subdomains from public sources

Overall, this reconnaissance phase provides a structured approach to understanding the target environment and lays the foundation for identifying vulnerabilities in later stages of the penetration testing process.

Scope and environment:

This project focuses on conducting a controlled penetration test against a deliberately vulnerable web application, Damn Vulnerable Web Application, hosted in a local lab environment.

The scope of the assessment includes:

- Target: http://localhost/DVWA
- Testing Type: Black-box / Gray-box (local controlled lab)

Activities Allowed:

- Reconnaissance and information gathering
- Vulnerability scanning and enumeration
- Exploitation of identified vulnerabilities within DVWA

Environment setup:
The penetration was set up on a local machine using the following setups:

- Operating System-Kali Linux (penetration testing distribution)
- Web Server-Apache-HTTP Server (running on port 80)
- Database-MariaDB (used as backend database for DVWA)
- Backend Technologies-PHP (server-side scripting)
- Vulnerable Application

Damn Vulnerable Web Application installed locally in:
/var/www/html/DVWA
Lab Configuration

- DVWA accessed via:http://localhost/DVWA
- Default credentials used:

  Username: admin
  Password: password
- Security level set to: Low-to allow demonstration of vulnerabilities in the lab.

The purpose of this environment is to provide a safe and controlled platform that simulates a real-world web application, allowing for practical penetration testing exercises. It enables the tester to apply, and practice established methodologies for reconnaissance, vulnerability identification, and exploitation in a legal and risk-free setting. By using a deliberately vulnerable application, the environment allows for hands-on exploration of common security flaws, while ensuring that no external systems or networks are affected, maintaining compliance with ethical and legal standards.

Target identification:

The target identified for this penetration testing assessment is the Damn Vulnerable Web Application, a deliberately vulnerable web application designed for security training and testing purposes. The application is hosted locally within a controlled lab environment and is accessible via the URL http://localhost/DVWA. Since the target resides on the local machine, it resolves the loopback IP address 127.0.0.1, ensuring that all testing activities remain internal and do not interact with external networks or systems.

The application is served using the Apache HTTP Server and utilizes PHP for server-side processing, with MariaDB functioning as the backend database. DVWA was selected as the target due to its intentionally insecure design, which provides a safe and legal platform for identifying and exploiting common web application vulnerabilities such as SQL injection, cross-site scripting (XSS), and file inclusion flaws. This makes it suitable for demonstrating penetration testing methodologies in a structured and controlled manner.

The identification of this target forms the foundation of the reconnaissance phase, enabling the tester to focus on information gathering, service enumeration, and vulnerability of discovery within a defined and authorized scope. By clearly defining the target as a local instance of DVWA, the assessment ensures compliance with ethical standards and avoids any unintended impact on external systems.

Methodology:

This project follows a structured penetration testing methodology applied to a controlled lab environment using DVWA as the target web application. The methodology is designed to simulate a real-world security assessment while ensuring that all activities are performed legally and safely. The workflow is divided into three main phases: reconnaissance, vulnerability analysis, and exploitation.

1. Reconnaissance (Passive and Active)

The first phase involves comprehensive reconnaissance to gather as much information about the target as possible. Reconnaissance is divided into passive and active methods:

- Passive Reconnaissance: This involves collecting information without directly interacting with the target system. Techniques include reviewing publicly available files such as robots.txt and sitemap.xml, analyzing HTTP headers to

determine server and software versions, and performing Open-Source Intelligence (OSINT) using tools like Wappalyzer to identify the technologies, frameworks, and platforms in use. Additionally, subdomain enumeration is conducted using OSINT databases to expand the attack surface and identify potential points of vulnerability.

- Active Reconnaissance: This involves controlled interaction with the target system to discover additional details and confirm assumptions made during passive reconnaissance. Tools such as Nmap are used for port scanning, Nikto for web server scanning, and directory brute-forcing tools such as Dirb, Gobuster, and ffuf are employed to discover hidden files, directories, and endpoints.

The reconnaissance phase establishes a foundation of knowledge about the target, which informs subsequent vulnerability identification and exploitation activities.

2. Vulnerability Analysis (Covered in Part 2)

The second phase involves identifying and analyzing vulnerabilities within the target system. This includes testing misconfigurations, weak authentication, input validation flaws, and other common web application vulnerabilities. The detailed procedures, findings, and analyses of this phase are presented in Part 2 of this project report, where each vulnerability will be documented along with evidence, impact, and reproducibility steps.

3. Exploitation

The final phase focuses on exploiting the discovered vulnerabilities in a safe and controlled manner to demonstrate the potential impact. Exploitation activities are limited to the local lab environment to ensure legal compliance. This phase tests the severity of vulnerabilities, confirms exploitability, and demonstrates practical penetration testing skills. Each exploitation step is documented carefully, providing insight into attack techniques and expected results.

Following this methodology, the project demonstrates an end-to-end penetration testing process starting from information gathering, moving through vulnerability identification, and concluding with controlled exploitation. This approach not only showcases technical proficiency but also emphasizes ethical and safe testing practices within a fully legal environment.

Findings:

Passive Reconnaissance:

Manual Discovery – Robots.txt

Robot.txt file is a document that tells search engines which pages they are and aren't allowed to be on their search engines.

The robots.txt file was accessed at http://localhost/DVWA/robots.txt and contains the following directives:

```
User-agent: *
Disallow: /
```

This indicates that all web crawlers are instructed not to access any part of the site. While this is intended to restrict indexing search engines, it also reveals that the site owner considers the entire application sensitive, which can guide further reconnaissance and enumeration in a controlled testing environment.

Sitemap.xml

It gives a list of every file the website the owner wishes to be listed on a search engine.

Not Found

The requested URL was not found on this server.

Apache/2.4.66 (Debian) Server at localhost Port 80

This indicates that the application does not have a publicly available sitemap, so no additional URLs or directories could be discovered using this method.

HTTP HEADERS

```
~ via 🐍 v3.13.12
❯ curl http://localhost/DVWA/ -v
* Host localhost:80 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:80...
* Established connection to localhost (::1 port 80) from ::1 port 35886
* using HTTP/1.x
> GET /DVWA/ HTTP/1.1
> Host: localhost
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 302 Found
< Date: Mon, 30 Mar 2026 20:16:33 GMT
< Server: Apache/2.4.66 (Debian)
< Set-Cookie: security=impossible; path=/; HttpOnly
< Set-Cookie: PHPSESSID=5f9efdaceefd389f29556104a50a1f4f; expires=Tue, 31 Mar 2026 20:16:33 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
< Expires: Thu, 19 Nov 1981 08:52:00 GMT
< Cache-Control: no-store, no-cache, must-revalidate
< Pragma: no-cache
< Set-Cookie: PHPSESSID=ec4effe9e5af9d52b3ba8d7a0a163000; expires=Tue, 31 Mar 2026 20:16:33 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
< Location: login.php
< Content-Length: 0
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host localhost:80 left intact
```

The curl command with the -v option was used to retrieve verbose output, including HTTP headers from the target application. The purpose of this analysis is to identify the underlying software and technologies used by the server.

The response revealed that the application is running on Apache/2.4.66 (Debian), providing insight into the web server in use. This information is valuable as it helps determine potential vulnerabilities associated with specific software versions. Additionally, session cookies such as PHPSESSID were observed, indicating active session management within the application.

Osint –open-source intelligence

Wappalyzer is an online tool that can be used to identify technologies a website is running.

Wappalyzer was used to identify the technologies used by the DVWA application. The tool detected that the site runs on Apache, uses PHP, and relies on MySQL/MariaDB for the database backend.

Understanding the technology stack helps penetration testers focus on vulnerabilities specific to these platforms, such as outdated PHP functions or misconfigured Apache settings.

Active Reconnaissance

Nmap:
A port scanning tool, such as Nmap, is a crucial component of the reconnaissance phase in penetration testing. It allows security testers to identify open, closed, and filtered ports on a target system. Each port state provides important information:

- Open ports – Services are actively listening and may be potential entry points for attacks.
- Closed ports – No service is listening, but the system is reachable; this can help map the network structure.
- Filtered ports – Traffic is blocked by a firewall or filtering device; this indicates security controls in place.

By discovering which ports are open and which services are running, testers can map the attack surface and prioritize areas for further vulnerability testing. Port scanning therefore forms the backbone of technical reconnaissance, bridging the gap between passive information gathering (like WHOIS or HTTP headers) and active exploitation

```
~ via 🐍 v3.13.12
❯ nmap -sS -sV -p- localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-30 23:08 +0200
Stats: 0:00:06 elapsed; 0 hosts completed (1 up), 1 undergoing Service Scan
Service scan Timing: About 0.00% done
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000030s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 65529 closed tcp ports (reset)
PORT     STATE SERVICE  VERSION
80/tcp   open  http     Apache httpd 2.4.66 ((Debian))
3306/tcp open  mysql?
8000/tcp open  http     Splunkd httpd
8065/tcp open  http     CherryPy wsgiserver
8089/tcp open  ssl/http Splunkd httpd
8191/tcp open  mongodb  MongoDB 3.6 after 3.6.3, or 3.7.3 or later
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port3306-TCP:V=7.98%I=7%D=3/30%Time=69CAE65A%P=x86_64-pc-linux-gnu%r(NU
SF:LL,64,"`\0\0\0\n11\.8\.6-MariaDB-4\x20from\x20Debian\0J\0\0\0\[V}2~H\)4
SF:\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0=\0\0\0\014f\)5nKi5D8,\0mysql_na
SF:tive_password\0")%r(GenericLines,9C,"`\0\0\0\n11\.8\.6-MariaDB-4\x20fro
SF:m\x20Debian\0J\0\0\0\[V}2~H\)4\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0
SF:=\0\0\014f\)5nKi5D8,\0mysql_native_password\x004\0\0\x01\xffj\x04#HY000
SF:Proxy\x20header\x20is\x20not\x20accepted\x20from\x20127\.0\.0\.1")%r(LD
SF:APBindReq,64,"`\0\0\0\n11\.8\.6-MariaDB-4\x20from\x20Debian\0\[`\0\0\0E3
SF:!lQ\]~%8\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0=\0\0\0\xgJrJBM:xrQ~\0m
SF:ysql_native_password\0")%r(afp,64,"`\0\0\0\n11\.8\.6-MariaDB-4\x20from\
SF:x20Debian\0e\0\0\0Bo\)yKGc\?\0\xfe\xff-\x02\0\xff\x81\x15\0\0\0\0\0=\
SF:0\0\0QV<j\"f\[xi\)\+:\0mysql_native_password\0");

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 30.44 seconds


~ via 🐍 v3.13.12 took 30s
```

An Nmap scan was performed on 127.0.0.1 using SYN scan (-sS) and service version detection (-sV) to identify open ports and running services. The results revealed multiple open ports, including 80 (HTTP) running Apache 2.4.66, 3306 (MySQL/MariaDB), and several additional HTTP services on ports 8000, 8065, and 8089, as well as 8191 (MongoDB).

The presence of multiple open ports indicates a broader attack surface, with each service representing a potential entry point for further vulnerability assessment. Identifying service versions is particularly useful for detecting known vulnerabilities associated with specific software.

Nikto:

is an open-source web server scanner that identifies potential vulnerabilities, outdated software, and misconfigurations. It gathers information by analyzing server responses, helping testers understand the server environment, and planning further security assessments.

```
Session  Actions  Edit  View  Help
---------------------------------------------------------------------------
+ Your Nikto installation is out of date.
+ Target IP:          127.0.0.1
+ Target Hostname:    localhost
+ Target Port:        80
+ Platform:           Linux/Unix
+ Start Time:         2026-03-30 23:24:15 (GMT2)
---------------------------------------------------------------------------
+ Server: Apache/2.4.66 (Debian)
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [013587] /DVWA/: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /DVWA/: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /DVWA/: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /DVWA/: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /DVWA/: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [750500] /DVWA/config/: Directory indexing found.
+ [000998] /DVWA/config/: Configuration information may be available remotely.
+ [750500] /DVWA/tests/: Directory indexing found.
+ [001896] /DVWA/tests/: This might be interesting.
+ [750500] /DVWA/database/: Directory indexing found.
+ [002727] /DVWA/database/: Database directory found.
+ [750500] /DVWA/docs/: Directory indexing found.
+ [006333] /DVWA/login.php: Admin login page/section found.
+ [006530] /DVWA/.git/index: Git Index file may contain directory listing information.
+ [006609] /DVWA/.git/HEAD: Git HEAD file found. Full repo details may be present.
+ [006807] /DVWA/.git/config: Git config file found. Infos about repo details may be present.

+ [007224] /DVWA/.gitignore: .gitignore file found. It is possible to grasp the directory structure.
+ [007279] /DVWA/.dockerignore: .dockerignore file found. It may be possible to grasp the directory structure and learn more about the site.
+ [007342] /DVWA/: X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive. See: https://devel
oper.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
+ [007352] /DVWA/: The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type.
See: https://www.netsparker.com/web-vulnerability-scanner/vulnerabilities/missing-content-type-header/
+ 7849 requests: 16 errors and 20 items reported on the remote host
+ End Time:           2026-03-30 23:29:43 (GMT2) (328 seconds)
---------------------------------------------------------------------------
+ 1 host(s) tested
```

The scan identified that the server is running Apache/2.4.66 (Debian) and revealed several security issues.

Multiple security headers (e.g., Content Security Policy, Strict-Transport-Security) were missing, which may expose the application to common web attacks. Additionally, directory indexing was enabled in several directories such as /config/, /tests/, and /database/, potentially exposing sensitive files.

The scan also discovered accessible resources including an admin login page, configuration directories, and exposed .git files, which could leak internal application information. These findings indicate weak server configuration and increase the attack surface for further exploitation.

Ffuf command:

```
~ via 🐍 v3.13.12
❯ ffuf -w /usr/share/seclists/Discovery/Web-Content/common.txt -u http://localhost/DVWA/FUZZ -mc 200,301,302

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://localhost/DVWA/FUZZ
 :: Wordlist         : FUZZ: /usr/share/seclists/Discovery/Web-Content/common.txt
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200,301,302
________________________________________________

.git/HEAD               [Status: 200, Size: 23, Words: 2, Lines: 2, Duration: 62ms]
.git/config             [Status: 200, Size: 262, Words: 19, Lines: 12, Duration: 63ms]
.git                    [Status: 301, Size: 350, Words: 21, Lines: 10, Duration: 90ms]
config                  [Status: 301, Size: 352, Words: 21, Lines: 10, Duration: 0ms]
database                [Status: 301, Size: 354, Words: 21, Lines: 10, Duration: 0ms]
docs                    [Status: 301, Size: 350, Words: 21, Lines: 10, Duration: 0ms]
external                [Status: 301, Size: 354, Words: 21, Lines: 10, Duration: 0ms]
favicon.ico             [Status: 200, Size: 1406, Words: 5, Lines: 2, Duration: 0ms]
.git/logs/              [Status: 200, Size: 1330, Words: 90, Lines: 20, Duration: 124ms]
index.php               [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 0ms]
php.ini                 [Status: 200, Size: 154, Words: 19, Lines: 6, Duration: 0ms]
phpinfo.php             [Status: 302, Size: 0, Words: 1, Lines: 1, Duration: 0ms]
.git/index              [Status: 200, Size: 27578, Words: 391, Lines: 144, Duration: 176ms]
.gitattributes          [Status: 200, Size: 474, Words: 47, Lines: 32, Duration: 183ms]
robots.txt              [Status: 200, Size: 25, Words: 3, Lines: 2, Duration: 0ms]
.gitignore              [Status: 200, Size: 293, Words: 28, Lines: 19, Duration: 195ms]
security.txt            [Status: 200, Size: 151, Words: 25, Lines: 2, Duration: 0ms]
tests                   [Status: 301, Size: 351, Words: 21, Lines: 10, Duration: 0ms]
:: Progress: [4750/4750] :: Job [1/1] :: 39 req/sec :: Duration: [0:00:05] :: Errors: 0 ::

~ via 🐍 v3.13.12 took 5s
❯
```

The scan successfully identified multiple accessible directories and files, including /config/, /database/, /docs/, /external/, and /tests/.

Additionally, several sensitive files and resources were discovered, such as php.ini, phpinfo.php, and robots.txt. Notably, exposed **.**git directories and file**s** (e.g., .git/config, .git/index) were identified, which may lead to disclosure of source code and internal repository information.

These findings indicate the presence of misconfigurations and exposed resources, significantly increasing the attack surface, and providing potential entry points for further exploitation.

Gobuster command;

```
~ via 🐍 v3.13.12 took 4s
❯ gobuster dir -u http://localhost/DVWA/ -w /usr/share/seclists/Discovery/Web-Content/common.txt
===============================================================
Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://localhost/DVWA/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/seclists/Discovery/Web-Content/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.8.2
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
.git                 (Status: 301) [Size: 350] [--> http://localhost/DVWA/.git/]
.git/HEAD            (Status: 200) [Size: 23]
.git/index           (Status: 200) [Size: 27578]
.gitattributes       (Status: 200) [Size: 474]
.gitignore           (Status: 200) [Size: 293]
.hta                 (Status: 403) [Size: 314]
.htaccess            (Status: 403) [Size: 314]
.git/config          (Status: 200) [Size: 262]
.htpasswd            (Status: 403) [Size: 314]
.git/logs/           (Status: 200) [Size: 1330]
config               (Status: 301) [Size: 352] [--> http://localhost/DVWA/config/]
database             (Status: 301) [Size: 354] [--> http://localhost/DVWA/database/]
docs                 (Status: 301) [Size: 350] [--> http://localhost/DVWA/docs/]
external             (Status: 301) [Size: 354] [--> http://localhost/DVWA/external/]
favicon.ico          (Status: 200) [Size: 1406]
index.php            (Status: 302) [Size: 0] [--> login.php]
php.ini              (Status: 200) [Size: 154]
phpinfo.php          (Status: 302) [Size: 0] [--> login.php]
robots.txt           (Status: 200) [Size: 25]
security.txt         (Status: 200) [Size: 151]
tests                (Status: 301) [Size: 351] [--> http://localhost/DVWA/tests/]
Progress: 4750 / 4750 (100.00%)
===============================================================
Finished
===============================================================
```

The scan identified multiple accessible directories, including /config/, /database/, /docs/, /external/, and /tests/.

Sensitive resources such as php.ini, robots.txt, and security.txt were also discovered. Notably, exposed. git directories and files (e.g.,. git/config,. git/index) were identified, which may lead to disclosure of source code and internal repository information.

Additionally, restricted files such as .htaccess and .htpasswd returned **403** Forbidden responses, indicating their presence but restricted access. These findings highlight misconfigurations and significantly increase the attack surface.

Dirb command;

```
~ via 🐍 v3.13.12
❯ dirb http://localhost/DVWA/ /usr/share/seclists/Discovery/Web-Content/common.txt

-----------------
DIRB v2.22
By The Dark Raver
-----------------

START_TIME: Mon Mar 30 23:40:13 2026
URL_BASE: http://localhost/DVWA/
WORDLIST_FILES: /usr/share/seclists/Discovery/Web-Content/common.txt

---------------------------------

GENERATED WORDS: 4748

---- Scanning URL: http://localhost/DVWA/ ----
==> DIRECTORY: http://localhost/DVWA/.git/
+ http://localhost/DVWA/.git/HEAD (CODE:200|SIZE:23)
+ http://localhost/DVWA/.git/config (CODE:200|SIZE:262)
+ http://localhost/DVWA/.git/index (CODE:200|SIZE:27578)
+ http://localhost/DVWA/.git/logs/ (CODE:200|SIZE:1330)
+ http://localhost/DVWA/.gitattributes (CODE:200|SIZE:474)
+ http://localhost/DVWA/.gitignore (CODE:200|SIZE:293)
==> DIRECTORY: http://localhost/DVWA/config/
==> DIRECTORY: http://localhost/DVWA/database/
==> DIRECTORY: http://localhost/DVWA/docs/
==> DIRECTORY: http://localhost/DVWA/external/
+ http://localhost/DVWA/favicon.ico (CODE:200|SIZE:1406)
+ http://localhost/DVWA/index.php (CODE:302|SIZE:0)
+ http://localhost/DVWA/php.ini (CODE:200|SIZE:154)
+ http://localhost/DVWA/phpinfo.php (CODE:302|SIZE:0)
+ http://localhost/DVWA/robots.txt (CODE:200|SIZE:25)
+ http://localhost/DVWA/security.txt (CODE:200|SIZE:151)
==> DIRECTORY: http://localhost/DVWA/tests/

---- Entering directory: http://localhost/DVWA/.git/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://localhost/DVWA/config/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://localhost/DVWA/database/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)

---- Entering directory: http://localhost/DVWA/docs/ ----
(!) WARNING: Directory IS LISTABLE. No need to scan it.
    (Use mode '-w' if you want to scan it anyway)
```

The scan identified multiple directories, including /config/, /database/, /docs/, /external/, and /tests/, many of which were listable, indicating that directory indexing is enabled.

Additionally, sensitive resources such as .git files (e.g., .git/config, .git/index), php.ini, and robots.txt were discovered. The presence of listable directories and exposed repository files increases the risk of information disclosure and provides valuable insight for further exploitation

Summary:

The first phase of the project focused on passive and active reconnaissance of the DVWA application hosted locally. Initial steps included reviewing robots.txt and attempting to locate a sitemap, which revealed restricted paths and disallowed content. HTTP headers were inspected using curl -v to identify the web server software (Apache 2.4.66) and session management details.

Port scanning with Nmap -sS -sV -p- localhost identified open services including HTTP (80), MySQL (3306), and other auxiliary services, providing insight into the application's attack surface.

Further enumeration using web content discovery tools (ffuf, gobuster, dirb) revealed multiple directories and files of interest, including:

- Listable directories: /config/, /database/, /docs/, /external/, /tests/
- Sensitive files:. git repository files, php.ini, robots.txt, and security.txt

These findings demonstrated potential information disclosure vulnerabilities and laid the groundwork for the exploitation phase. The methodology combines manual exploration, HTTP inspection, port scanning, and automated directory/file enumeration to comprehensively map the DVWA environment.